DPDzero Data Ops Pipeline Documentation

Overview

The **DPDzero Data Ops Pipeline** processes daily loan collection call data, generating performance metrics and insights. The pipeline includes data ingestion, merging, feature engineering, and reporting. The data is processed from multiple sources:

- **Call Logs**: Contains details of individual call attempts.
- **Agent Roster**: Contains metadata about the agents (name, office location, etc.).
- **Disposition Summary**: Contains information about agent logins for a given date.

1. Data Ingestion and Validation (ingestion.py)

This module is responsible for reading the CSV files and ensuring that all required columns are present and properly formatted. It also handles missing or duplicate entries.

Key Functions:

read_csv_with_validation(file_path, expected_columns, key_columns)

- **Parameters:**
  - file_path: Path to the CSV file to be read.
  - expected_columns: List of columns that are expected to be present in the file.
  - key_columns: List of columns that should not contain missing or null values.
- **Returns: A pandas DataFrame containing the validated data.**

  Process:
  - Reads the CSV file.
  - Checks if the expected columns are present and logs missing columns.
  - Flags and logs missing or null values in required key columns.
  - Drops duplicate rows and logs the number of rows dropped.
  - Returns the cleaned DataFrame.

2. Data Merging (merge.py)

This module is responsible for merging the datasets (call logs, agent roster, and disposition summary) on agent_id, org_id, and call_date.

Key Functions:

merge_data(call_logs, agent_roster, disposition_summary)

- **Parameters**:
  - call_logs: DataFrame containing the call logs.
  - agent_roster: DataFrame containing the agent roster.
  - disposition_summary: DataFrame containing the disposition summary.
- **Returns: A merged DataFrame containing information from all three sources.**

  Process:
  - Converts call_date and login_time to proper datetime formats.
  - Merges the datasets using left joins, ensuring no data loss.
  - Logs warnings if there are agents in the call logs that cannot be found in the agent roster.

3. Feature Engineering (features.py)

This module computes key metrics such as the number of calls, unique loans contacted, connect rate, and average call duration for each agent on a given day.

Key Functions:

compute_metrics(df)

- **Parameters:**
  - df: The merged DataFrame containing call logs, agent information, and disposition summary.
  - Returns: A DataFrame summarizing agent performance metrics.

  Process:
  - Calculates the following:
  - Total Calls: The total number of calls made by an agent.
  - Unique Loans Contacted: The number of unique loans contacted by an agent.
  - Connect Rate: The percentage of calls that were marked as completed (successful calls).
  - Avg Call Duration: The average duration of calls in minutes.
  - Presence: Whether the agent was present (logged in) for that day.
  - Groups the data by agent_id, call_date, and other relevant features to compute these metrics.
- **Returns a DataFrame with aggregated performance data.**

4. Report Generation (reporting.py)

This module generates a Slack-style summary message based on the computed metrics.

Key Functions:

generate_slack_summary(summary_df, report_date)

- **Parameters:**
  - summary_df: DataFrame containing the computed performance metrics for agents.
  - report_date: The date for which the report is to be generated.
- **Returns: A formatted string containing a summary of the top performer, total active agents, and average call duration.**

  Process:

  - Filters the performance summary for the given report_date.
  - Identifies the top performer based on connect rate.
  - Calculates and logs the average call duration and the number of active agents.
  - Formats the report into a readable message, including the top performer's details, total active agents, and average duration.

5. Main Script (main.py)

The main.py script orchestrates the entire pipeline. It reads the input files, processes the data through ingestion, merging, feature engineering, and reporting, and finally saves the results.

Key Functions:

main()

Process:

- Reads the CSV files using the read_csv_with_validation function.
- Merges the datasets using the merge_data function.
- Computes performance metrics using the compute_metrics function.
- Generates a summary report using the generate_slack_summary function.
- Saves the final report as agent_perform

How to Run the Pipeline

Set up the Environment:

Install the necessary Python dependencies: pandas

pip install pandas

Prepare the Input Files:

Ensure the following CSV files are available in the correct format:

- call_logs.csv
- agent_roster.csv
- disposition_summary.csv

Run the Pipeline:

Execute the pipeline from the command line by running:

python main.py --call_logs data/call_logs.csv --agent_roster data/agent_roster.csv --disposition_summary data/disposition_summary.csv

The output will be saved as agent_performance_summary.csv and will include a Slack-style report for the specified data.